

#1.1. Write a query to display all customers from the city 'Delhi'.

SQL File 9*

Limit to 1000 rows

1 • `SELECT * FROM customers WHERE city = 'Delhi';`

Result Grid

	customer_id	full_name	email	phone_number	city	signup_date	signup_date_new
▶	512203	Tushar Rau	lmangat@yahoo.com	8215928685	Delhi	08-02-2024	2024-02-08
	457035	Jhanvi Sehgal	darshitbose@sridhar.biz	8745916414	Delhi	24-03-2025	2025-03-24
	597401	Kiaan Bansal		7766959137	Delhi	20-03-2025	2025-03-20
	462199	Tara Dani	cbajaj@hotmail.com	9559381066	Delhi	11-04-2025	2025-04-11
	903347	Trisha Ghosh	aaryahi77@yahoo.com	9403455972	Delhi	13-02-2025	2025-02-13
	643914	Kiara Golla		8620200747	Delhi	04-05-2024	2024-05-04
	712034	Divyansh Bhasin	jivikavenkatesh@gmail.com	9243605829	Delhi	19-03-2025	2025-03-19
	530489	Alia Deo	kartikgoswami@sule-divan.info	9838248510	Delhi	14-03-2024	2024-03-14
	318287	Jiya Mann		9012500763	Delhi	22-03-2025	2025-03-22
	773075	Navya Dada	esuresh@dada.com	7104332746	Delhi	28-07-2023	2023-07-28
	813895	Devansh Sethi		8650542480	Delhi	04-05-2023	2023-05-04
	861486	Heer Shah	frandhawa@hotmail.com	8080284453	Delhi	28-06-2024	2024-06-28
	345010	Kabir Verma	shinde@shinde.com	8401731470	Delhi	11-05-2023	2023-05-11

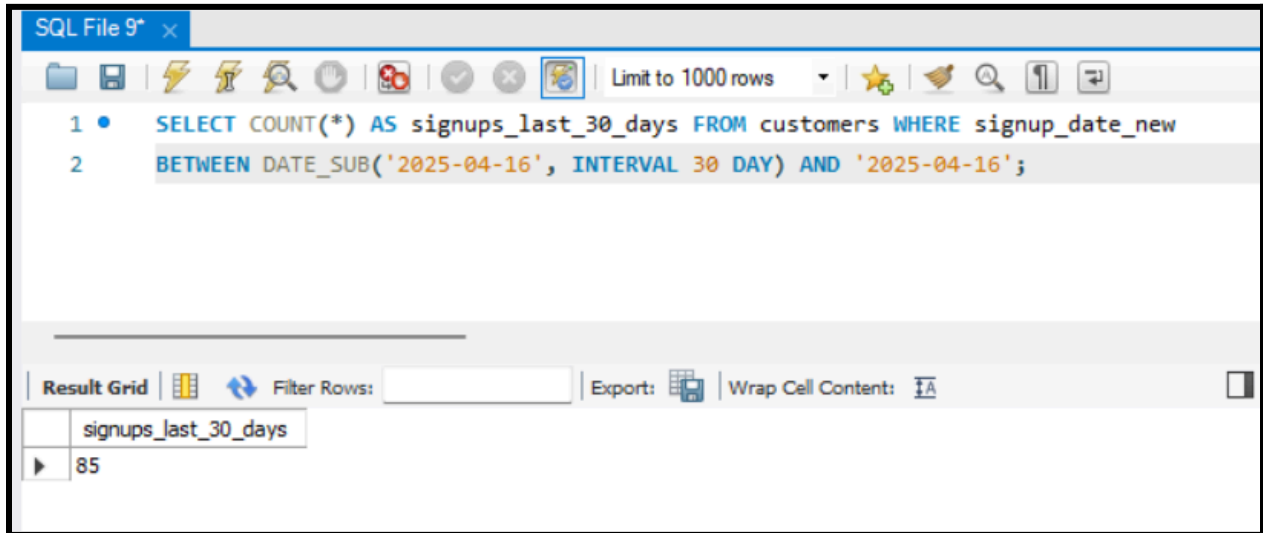
customers 7

Output

Action Output

#	Time	Action
✓ 137	23:07:49	use xeno_assignment
✓ 138	23:08:14	ALTER TABLE customers ADD COLUMN signup_date_new DATE
✓ 139	23:08:35	SET SQL_SAFE_UPDATES = 0
✓ 140	23:08:55	UPDATE customers SET signup_date_new = STR_TO_DATE(signup_date, '%d-%m-%Y')
✓ 141	23:09:06	SET SQL_SAFE_UPDATES = 1
✓ 142	23:10:14	SELECT * FROM customers WHERE city = 'Delhi' LIMIT 0, 1000

#1.2. Count the number of signups in the last 30 days. Assume today to be 16th April 2025



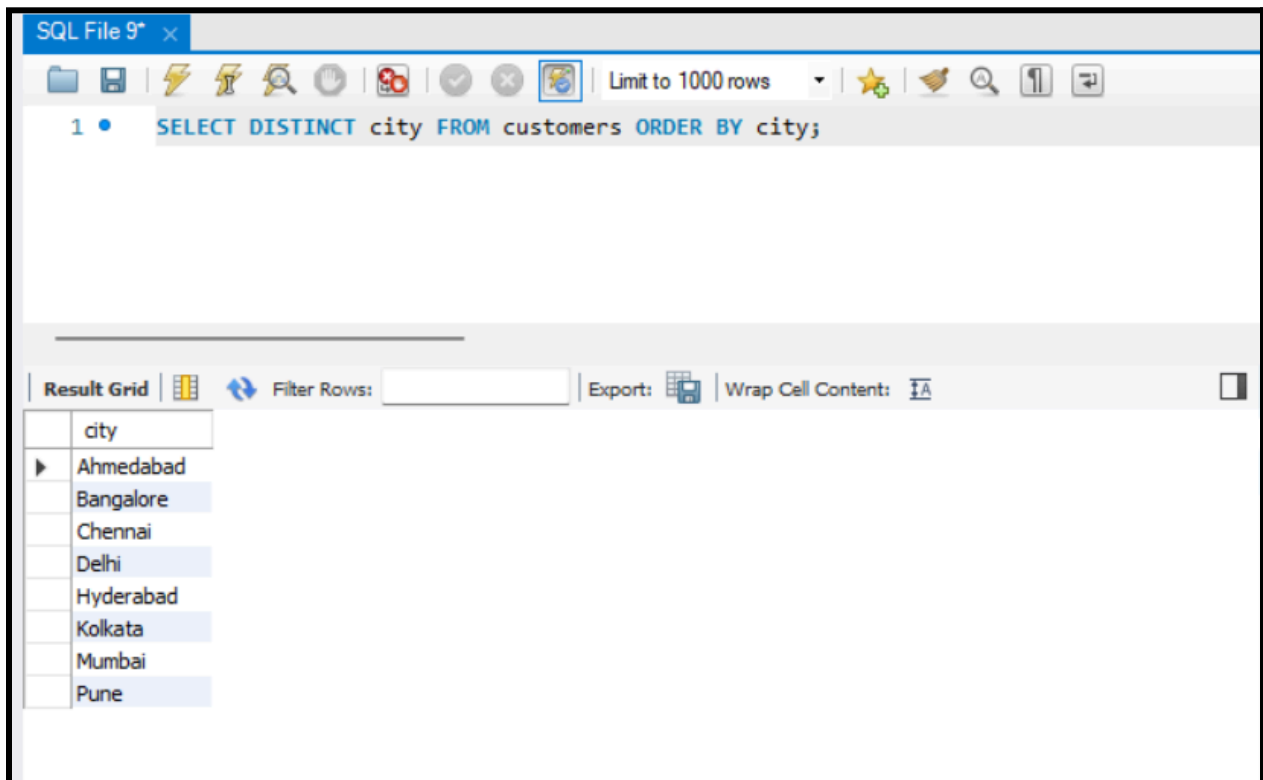
The screenshot shows a SQL editor window titled "SQL File 9*" with a toolbar and a query editor. The query is as follows:

```
1 • SELECT COUNT(*) AS signups_last_30_days FROM customers WHERE signup_date_new  
2 BETWEEN DATE_SUB('2025-04-16', INTERVAL 30 DAY) AND '2025-04-16';
```

Below the query editor, there is a "Result Grid" section with a "Filter Rows" input field, an "Export" button, and a "Wrap Cell Content" checkbox. The result grid shows a single row with the column name "signups_last_30_days" and the value "85".

signups_last_30_days
85

#1.3. List unique cities where customers are based



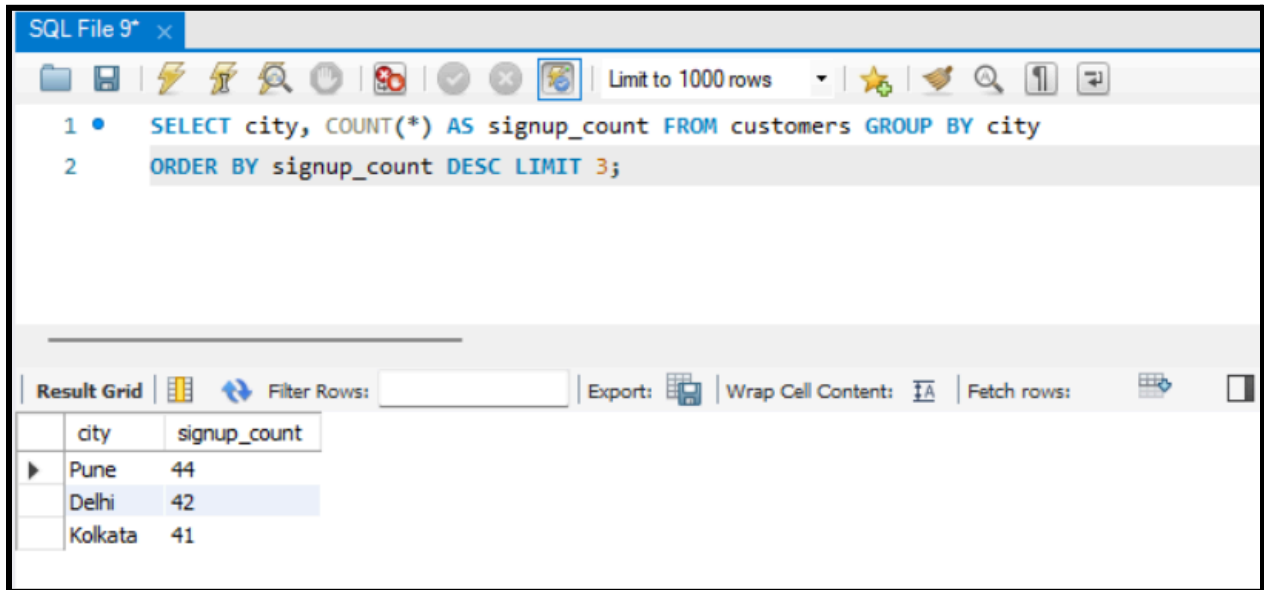
The screenshot shows a SQL editor window titled "SQL File 9*" with a toolbar and a query editor. The query is as follows:

```
1 • SELECT DISTINCT city FROM customers ORDER BY city;
```

Below the query editor, there is a "Result Grid" section with a "Filter Rows" input field, an "Export" button, and a "Wrap Cell Content" checkbox. The result grid shows a list of unique cities: Ahmedabad, Bangalore, Chennai, Delhi, Hyderabad, Kolkata, Mumbai, and Pune.

city
Ahmedabad
Bangalore
Chennai
Delhi
Hyderabad
Kolkata
Mumbai
Pune

#1.4. List the top 3 cities by number of signups.



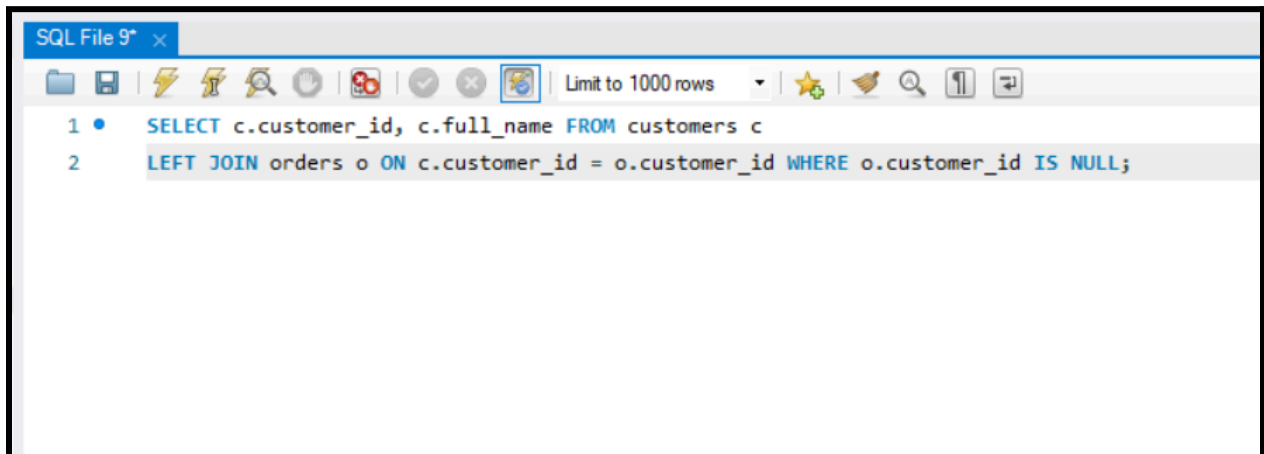
The screenshot shows a SQL editor window titled "SQL File 9*" with a toolbar and a "Limit to 1000 rows" dropdown. The query is as follows:

```
1 • SELECT city, COUNT(*) AS signup_count FROM customers GROUP BY city
2 ORDER BY signup_count DESC LIMIT 3;
```

Below the query, the "Result Grid" tab is active, displaying the following data:

city	signup_count
Pune	44
Delhi	42
Kolkata	41

#1.5. Assume there's another table order (customer_id, order_id, amount). How would you find customers who have never placed an order?

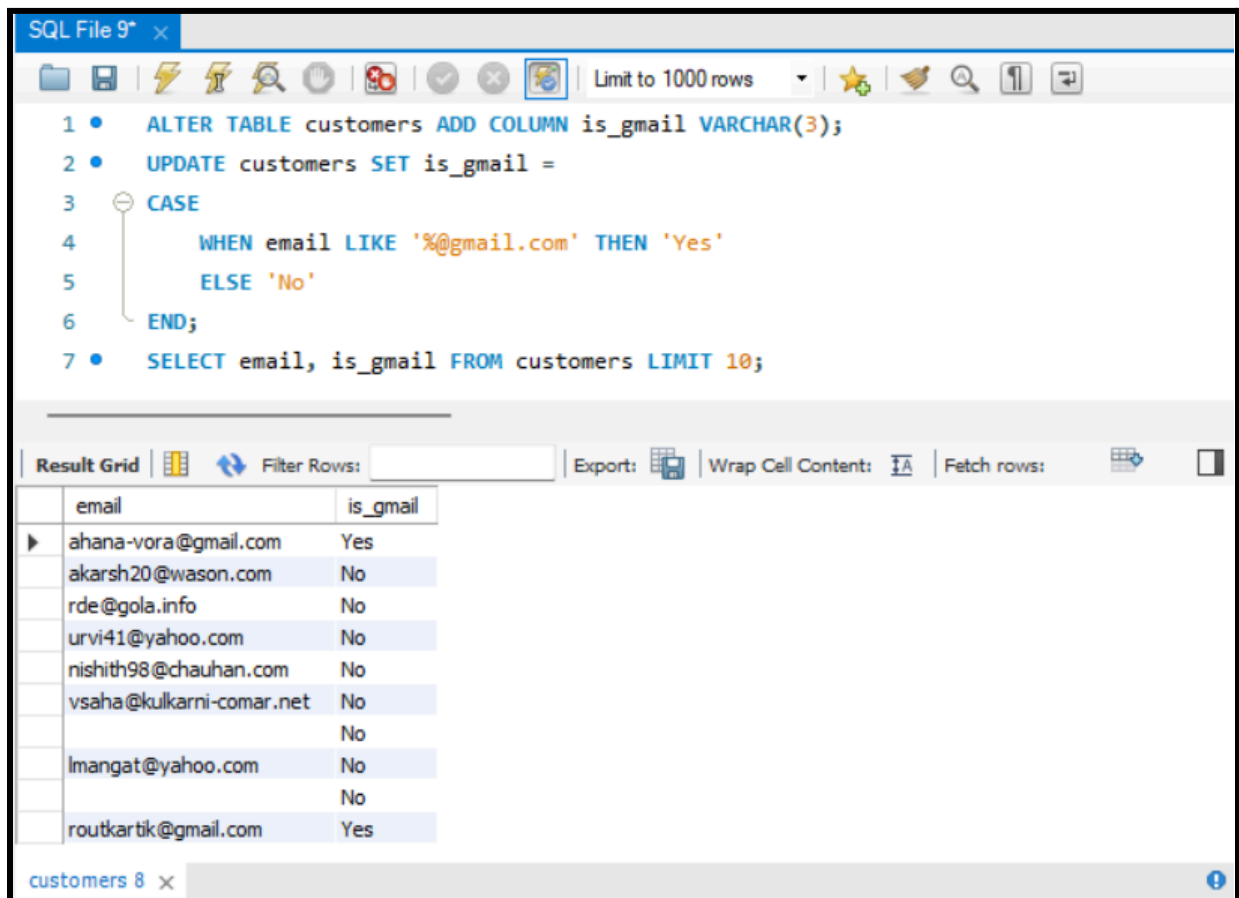


The screenshot shows a SQL editor window titled "SQL File 9*" with a toolbar and a "Limit to 1000 rows" dropdown. The query is as follows:

```
1 • SELECT c.customer_id, c.full_name FROM customers c
2 LEFT JOIN orders o ON c.customer_id = o.customer_id WHERE o.customer_id IS NULL;
```

PART 2 - Data Transformation & Enrichment

#2.1. Add a new column to show if the email domain is 'gmail.com' or not. Fill it with 'Yes' or 'No'.



The screenshot shows a SQL IDE window titled "SQL File 9* x". The query editor contains the following SQL code:

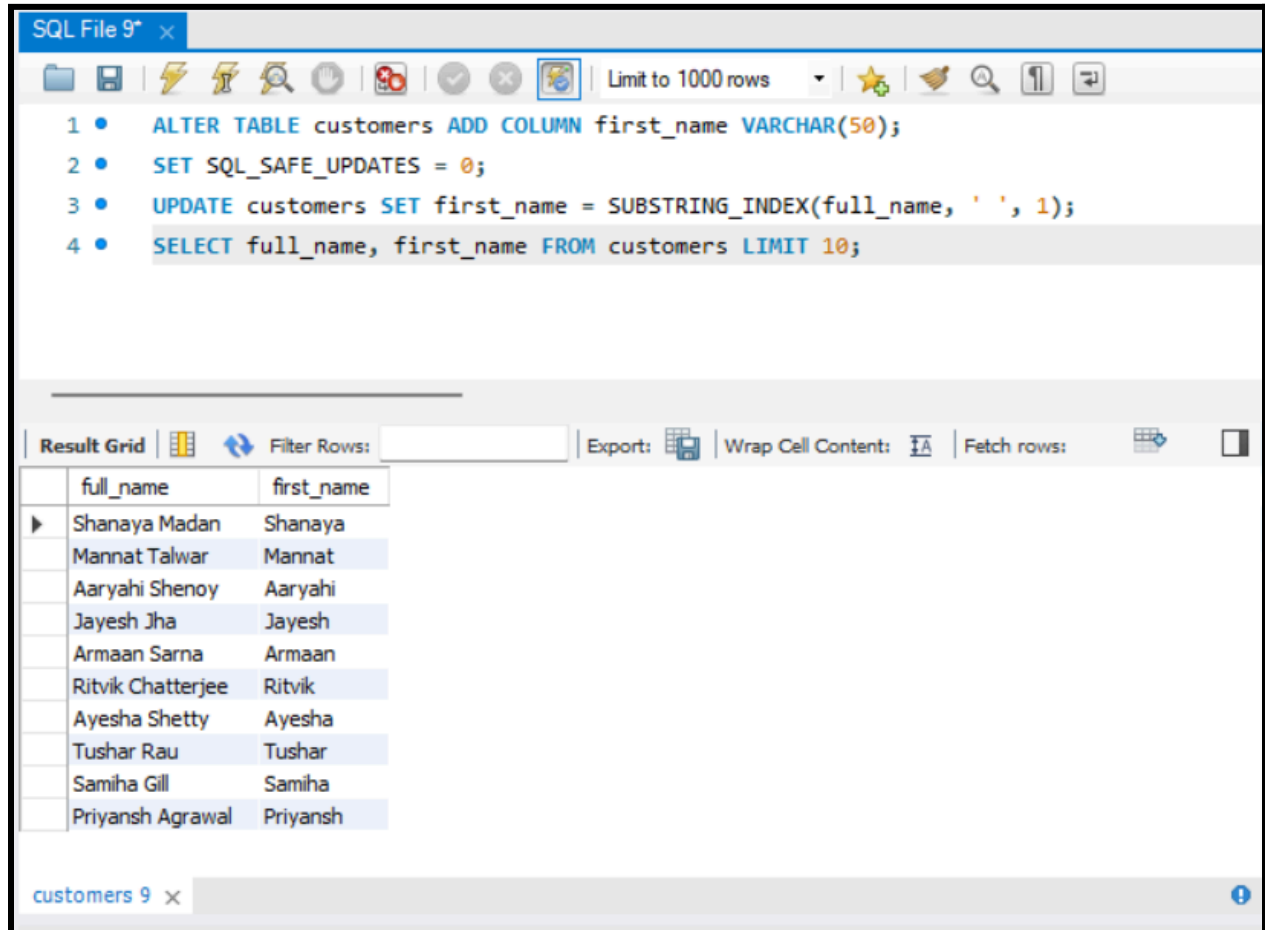
```
1 • ALTER TABLE customers ADD COLUMN is_gmail VARCHAR(3);
2 • UPDATE customers SET is_gmail =
3 • CASE
4 •     WHEN email LIKE '%@gmail.com' THEN 'Yes'
5 •     ELSE 'No'
6 • END;
7 • SELECT email, is_gmail FROM customers LIMIT 10;
```

The result grid shows the following data:

email	is_gmail
ahana-vora@gmail.com	Yes
akarsh20@wason.com	No
rde@gola.info	No
urvi41@yahoo.com	No
nishith98@chauhan.com	No
vsaha@kulkarni-comar.net	No
	No
lmangat@yahoo.com	No
	No
routkartik@gmail.com	Yes

The status bar at the bottom indicates "customers 8 x".

#2.2. Extract the first name from the name column and store it in 'first_name'.



The screenshot shows a SQL editor window titled "SQL File 9*" with a toolbar and a list of SQL commands. Below the editor is a "Result Grid" showing the output of the SQL commands. The result grid has two columns: "full_name" and "first_name". The data is as follows:

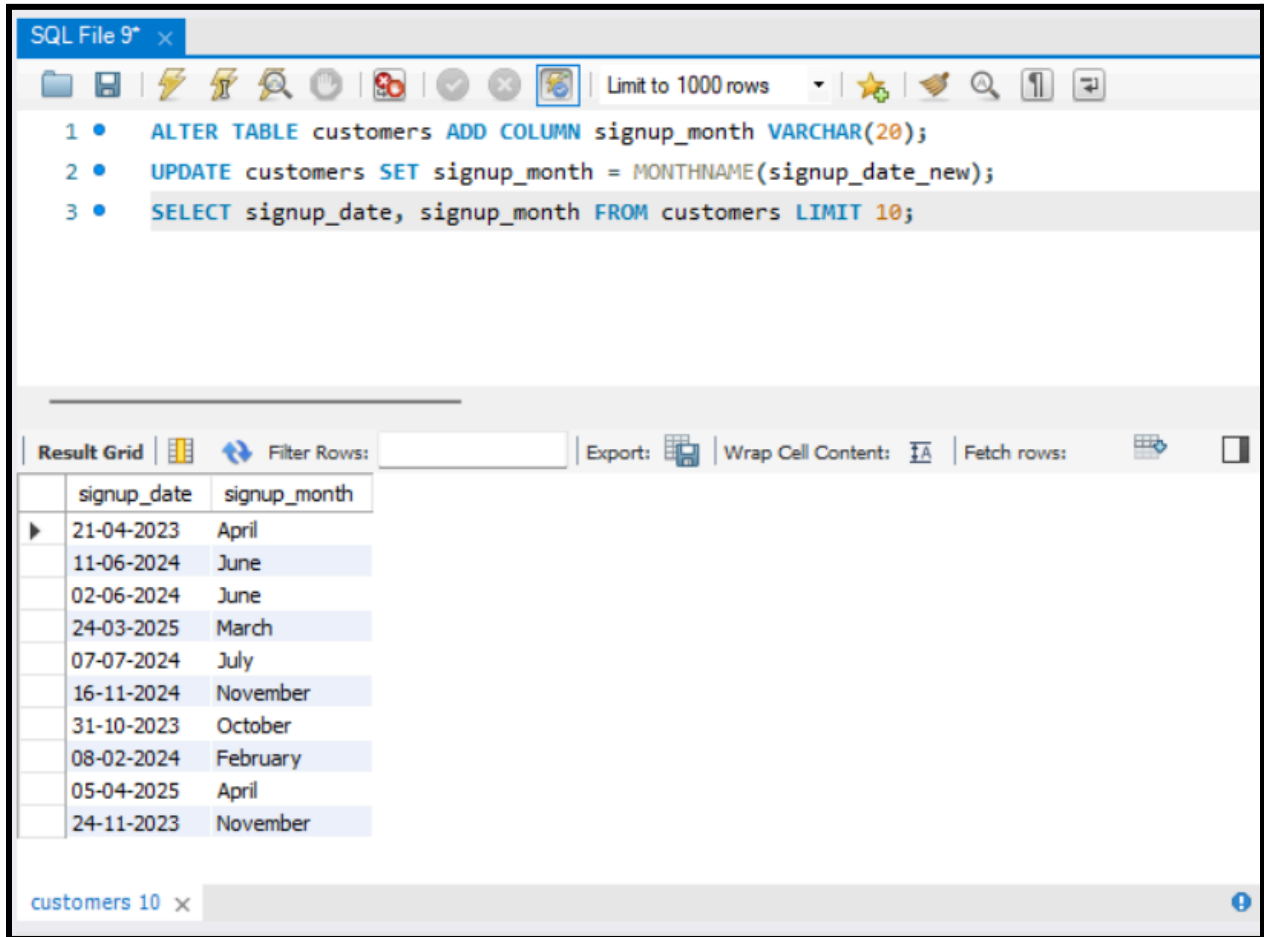
full_name	first_name
Shanaya Madan	Shanaya
Mannat Talwar	Mannat
Aaryahi Shenoy	Aaryahi
Jayesh Jha	Jayesh
Armaan Sarna	Armaan
Ritvik Chatterjee	Ritvik
Ayesha Shetty	Ayesha
Tushar Rau	Tushar
Samiha Gill	Samiha
Priyansh Agrawal	Priyansh

The SQL commands in the editor are:

```
1 • ALTER TABLE customers ADD COLUMN first_name VARCHAR(50);
2 • SET SQL_SAFE_UPDATES = 0;
3 • UPDATE customers SET first_name = SUBSTRING_INDEX(full_name, ' ', 1);
4 • SELECT full_name, first_name FROM customers LIMIT 10;
```

The bottom of the window shows a tab labeled "customers 9" and a status bar with an information icon.

#2.3. Add a column 'signup_month' to capture the month name the customer signed up.



The screenshot shows a SQL IDE window titled "SQL File 9* x". The toolbar includes icons for file operations, execution, and a "Limit to 1000 rows" dropdown. The SQL editor contains three commands:

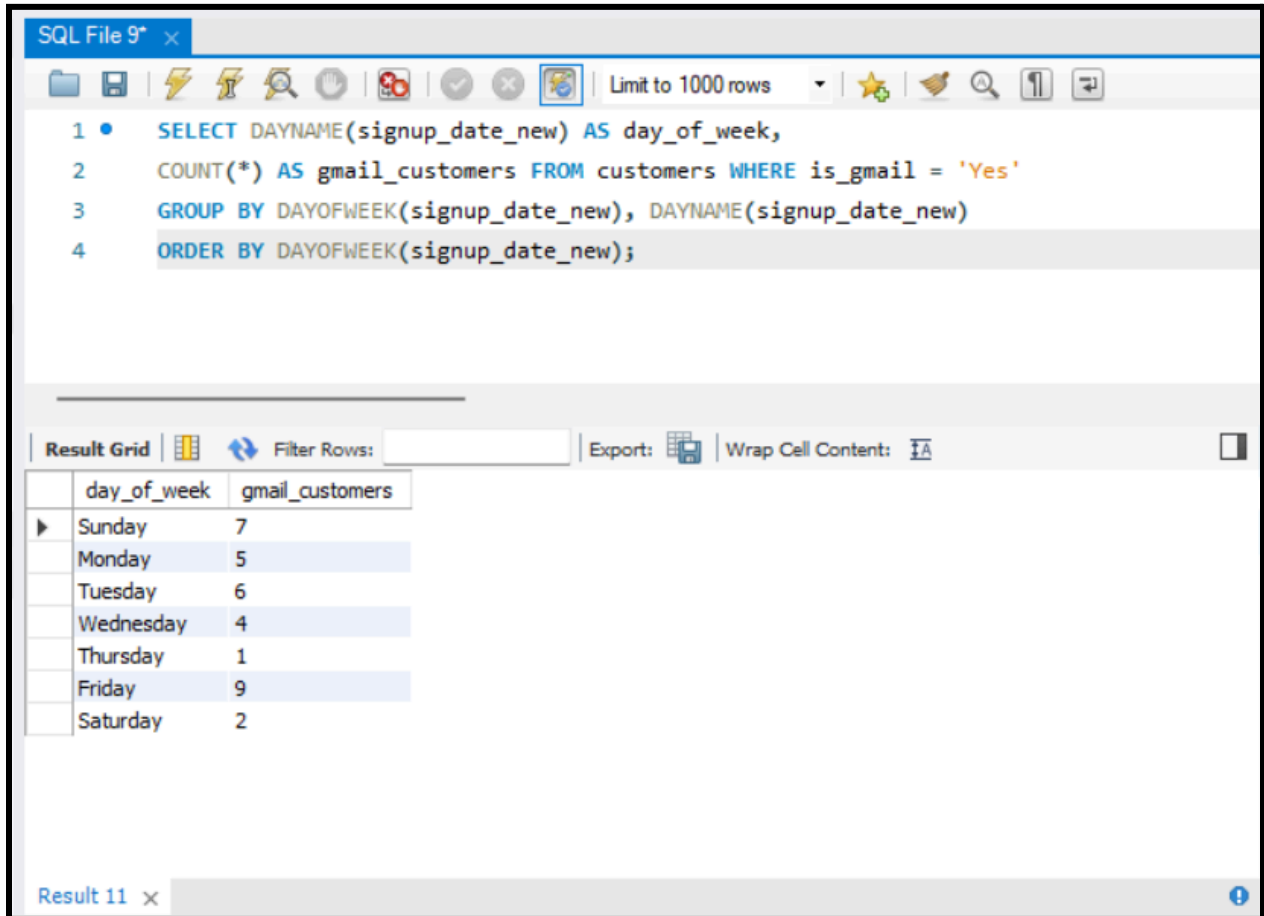
- 1 • `ALTER TABLE customers ADD COLUMN signup_month VARCHAR(20);`
- 2 • `UPDATE customers SET signup_month = MONTHNAME(signup_date_new);`
- 3 • `SELECT signup_date, signup_month FROM customers LIMIT 10;`

Below the editor is the "Result Grid" section, which includes a "Filter Rows:" input field and buttons for "Export:", "Wrap Cell Content:", and "Fetch rows:". The grid displays the results of the third command, showing 10 rows of data with columns "signup_date" and "signup_month".

signup_date	signup_month
21-04-2023	April
11-06-2024	June
02-06-2024	June
24-03-2025	March
07-07-2024	July
16-11-2024	November
31-10-2023	October
08-02-2024	February
05-04-2025	April
24-11-2023	November

The status bar at the bottom indicates "customers 10 x" and includes a help icon.

#2.4. Create a report which shows the no. of GMAIL customers who have signed up for each day of the week.



The screenshot shows a SQL IDE window titled "SQL File 9* x". The query editor contains the following SQL code:

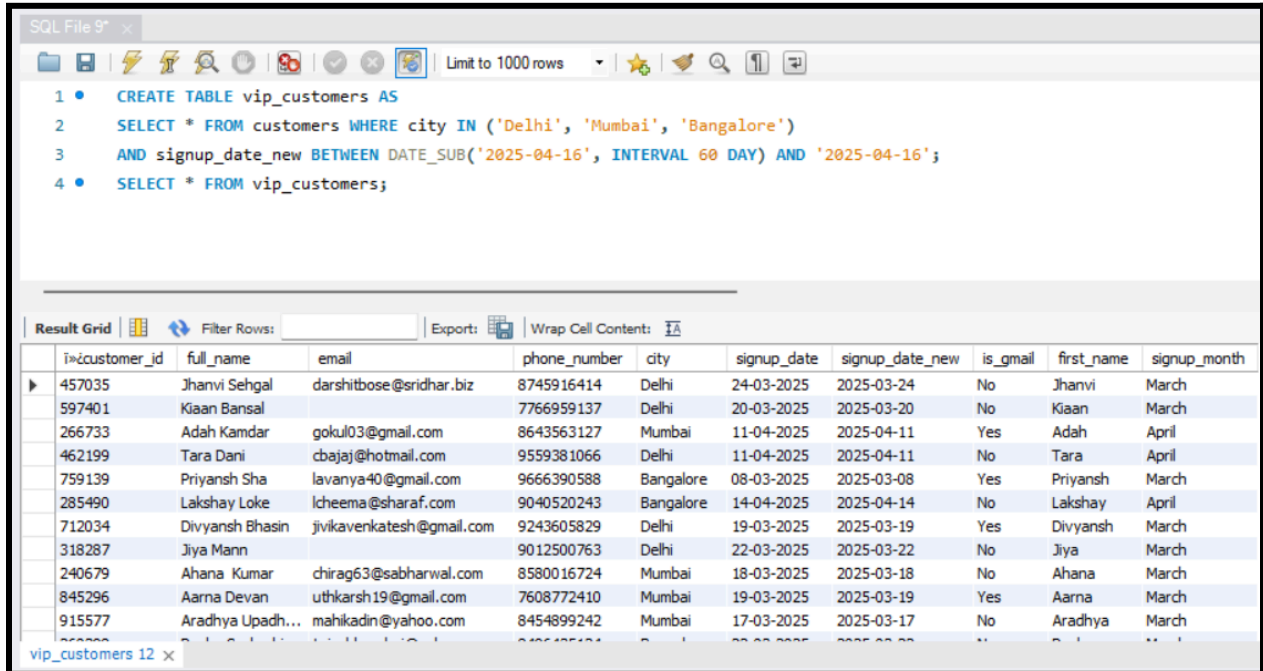
```
1 • SELECT DAYNAME(signup_date_new) AS day_of_week,  
2 COUNT(*) AS gmail_customers FROM customers WHERE is_gmail = 'Yes'  
3 GROUP BY DAYOFWEEK(signup_date_new), DAYNAME(signup_date_new)  
4 ORDER BY DAYOFWEEK(signup_date_new);
```

Below the query editor, the "Result Grid" tab is active, displaying the results of the query. The results are as follows:

day_of_week	gmail_customers
Sunday	7
Monday	5
Tuesday	6
Wednesday	4
Thursday	1
Friday	9
Saturday	2

The IDE also shows a toolbar with various icons, a "Limit to 1000 rows" dropdown, and a "Filter Rows:" input field. The status bar at the bottom indicates "Result 11 x" and a help icon.

#2.5. Create a new table 'vip_customers' with customers from Delhi, Mumbai and Bangalore who have signed up in the last 60 days from 16th April 2025.

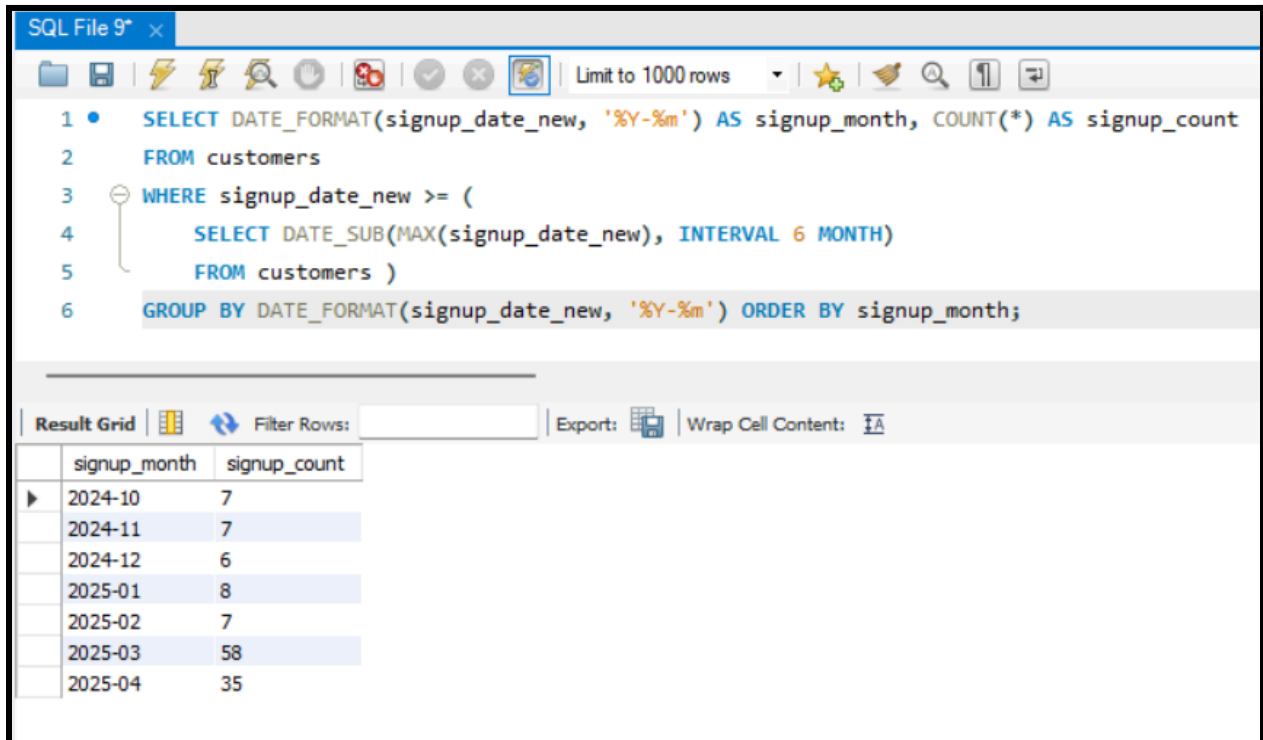


The screenshot shows a SQL IDE interface. The top pane contains a SQL query with four lines: 1. CREATE TABLE vip_customers AS; 2. SELECT * FROM customers WHERE city IN ('Delhi', 'Mumbai', 'Bangalore'); 3. AND signup_date_new BETWEEN DATE_SUB('2025-04-16', INTERVAL 60 DAY) AND '2025-04-16'; 4. SELECT * FROM vip_customers;. The bottom pane shows the 'Result Grid' with 11 columns: customer_id, full_name, email, phone_number, city, signup_date, signup_date_new, is_gmail, first_name, and signup_month. The grid contains 12 rows of data for customers from Delhi, Mumbai, and Bangalore who signed up between March 24 and April 14, 2025.

customer_id	full_name	email	phone_number	city	signup_date	signup_date_new	is_gmail	first_name	signup_month
457035	Jhanvi Sehgal	darshitbose@sridhar.biz	8745916414	Delhi	24-03-2025	2025-03-24	No	Jhanvi	March
597401	Kiaan Bansal		7766959137	Delhi	20-03-2025	2025-03-20	No	Kiaan	March
266733	Adah Kamdar	gokul03@gmail.com	8643563127	Mumbai	11-04-2025	2025-04-11	Yes	Adah	April
462199	Tara Dani	cbajaj@hotmail.com	9559381066	Delhi	11-04-2025	2025-04-11	No	Tara	April
759139	Priyansh Sha	lavanya40@gmail.com	9666390588	Bangalore	08-03-2025	2025-03-08	Yes	Priyansh	March
285490	Lakshay Loke	lcheema@sharaf.com	9040520243	Bangalore	14-04-2025	2025-04-14	No	Lakshay	April
712034	Divyansh Bhasin	jivikavenkatesh@gmail.com	9243605829	Delhi	19-03-2025	2025-03-19	Yes	Divyansh	March
318287	Jiya Mann		9012500763	Delhi	22-03-2025	2025-03-22	No	Jiya	March
240679	Ahana Kumar	chirag63@sabharwal.com	8580016724	Mumbai	18-03-2025	2025-03-18	No	Ahana	March
845296	Aarna Devan	uthkarsh19@gmail.com	7608772410	Mumbai	19-03-2025	2025-03-19	Yes	Aarna	March
915577	Aradhya Upadhyay	mahikadin@yahoo.com	8454899242	Mumbai	17-03-2025	2025-03-17	No	Aradhya	March

PART 3 - Analytics & Reporting

#3.1. Show a monthly signup count for the past 6 months.



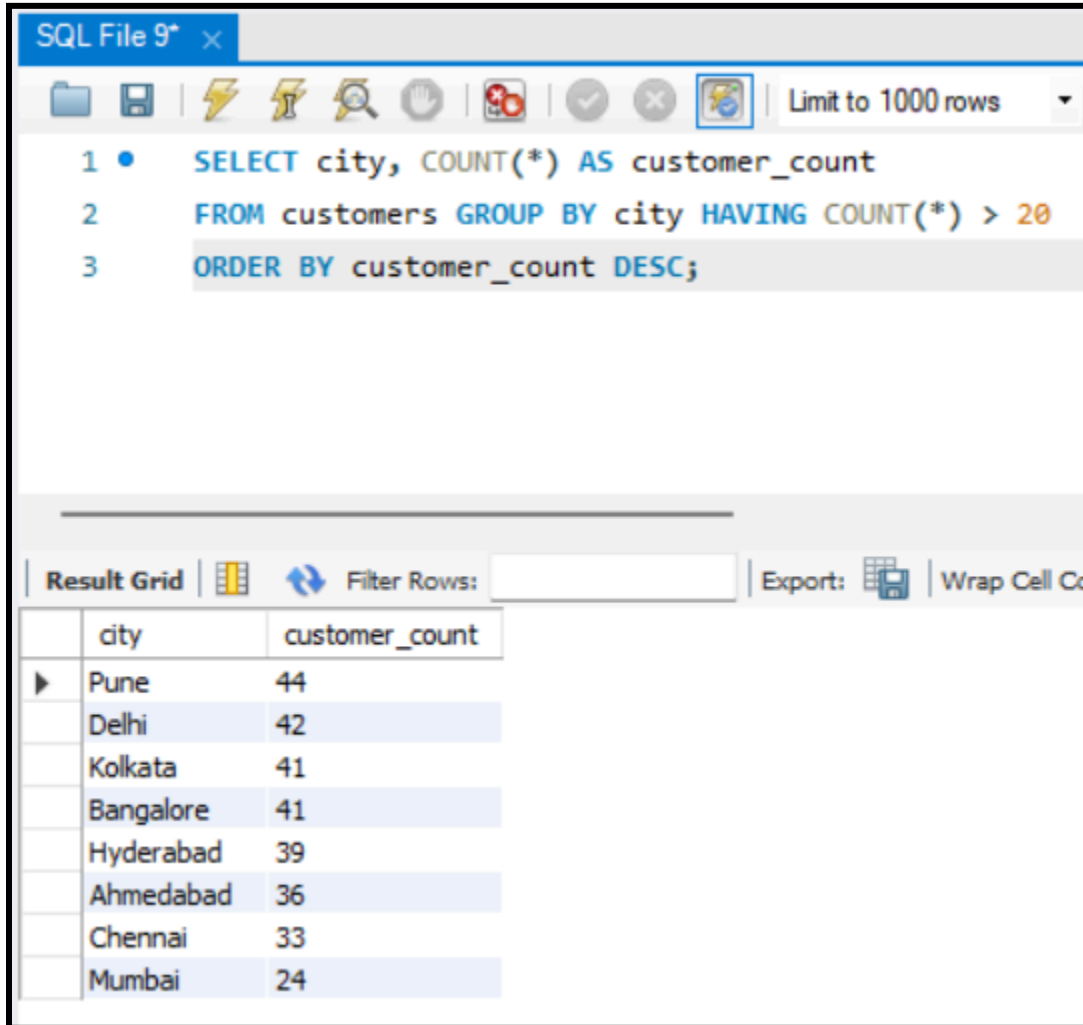
The screenshot shows a SQL IDE window titled "SQL File 9* x". The query editor contains the following SQL code:

```
1 • SELECT DATE_FORMAT(signup_date_new, '%Y-%m') AS signup_month, COUNT(*) AS signup_count
2 FROM customers
3 WHERE signup_date_new >= (
4     SELECT DATE_SUB(MAX(signup_date_new), INTERVAL 6 MONTH)
5     FROM customers )
6 GROUP BY DATE_FORMAT(signup_date_new, '%Y-%m') ORDER BY signup_month;
```

Below the query editor, the "Result Grid" tab is active, displaying the results of the query. The results are as follows:

signup_month	signup_count
2024-10	7
2024-11	7
2024-12	6
2025-01	8
2025-02	7
2025-03	58
2025-04	35

#3.2. Get a list of cities with more than 20 customers.



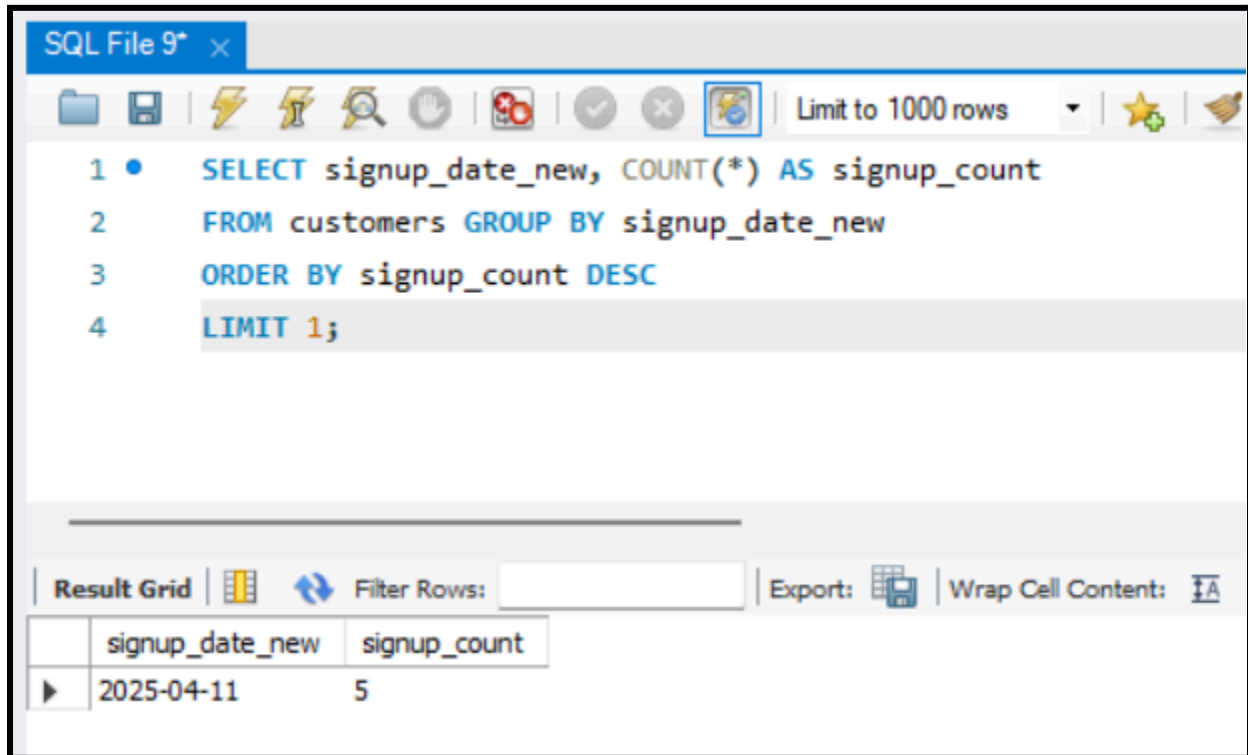
The screenshot shows an SQL IDE window titled "SQL File 9* x". The query editor contains the following SQL query:

```
1 • SELECT city, COUNT(*) AS customer_count
2 FROM customers GROUP BY city HAVING COUNT(*) > 20
3 ORDER BY customer_count DESC;
```

Below the query editor, the "Result Grid" tab is active, displaying the results of the query in a table. The table has two columns: "city" and "customer_count". The results are sorted in descending order of customer count.

city	customer_count
Pune	44
Delhi	42
Kolkata	41
Bangalore	41
Hyderabad	39
Ahmedabad	36
Chennai	33
Mumbai	24

#3.3. Find the date with the highest number of signups.



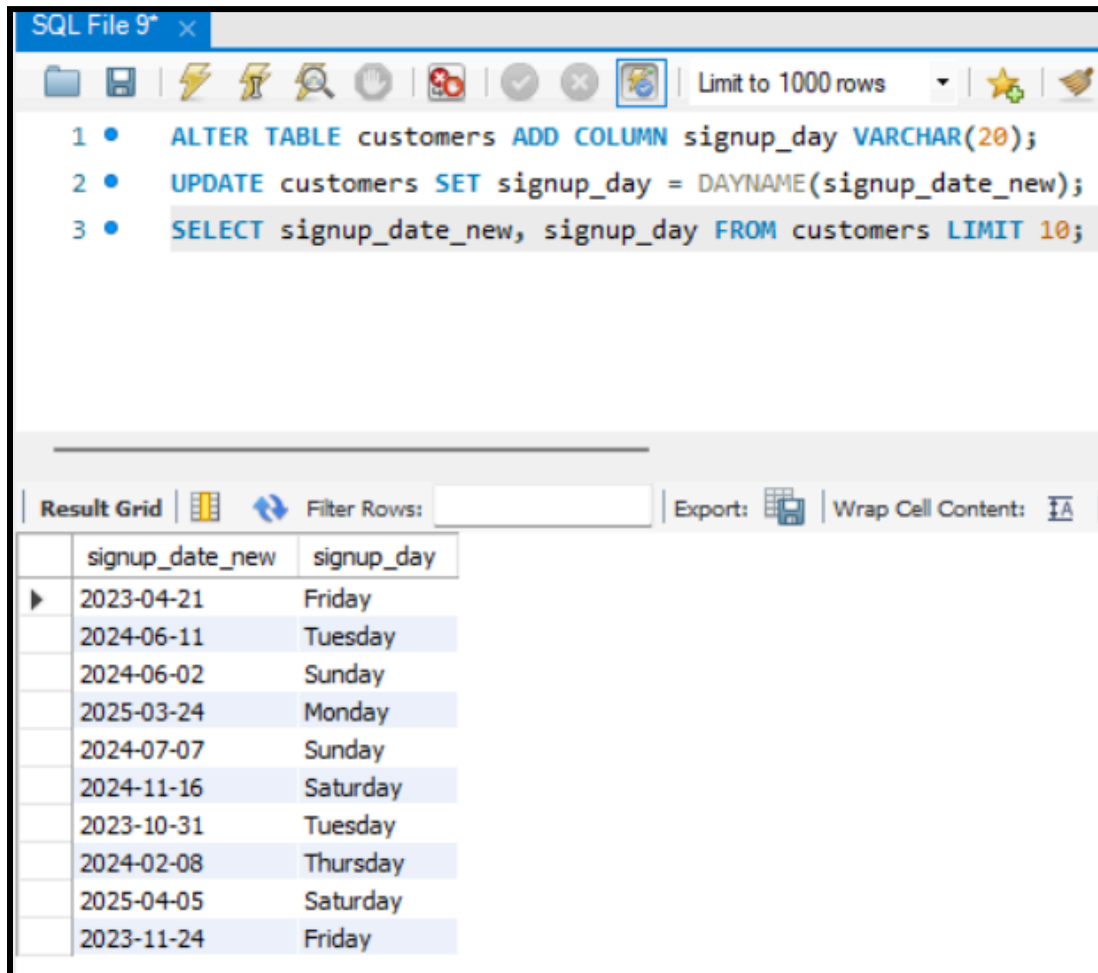
The screenshot shows a SQL IDE window titled "SQL File 9* x". The query editor contains the following SQL code:

```
1 • SELECT signup_date_new, COUNT(*) AS signup_count
2 FROM customers GROUP BY signup_date_new
3 ORDER BY signup_count DESC
4 LIMIT 1;
```

Below the query editor, the "Result Grid" tab is active. It displays a table with two columns: "signup_date_new" and "signup_count". The first row of data shows the date "2025-04-11" with a count of "5".

signup_date_new	signup_count
2025-04-11	5

#3.4. Add a new column to show the day of the signup date. Find the day with the highest number of signups.

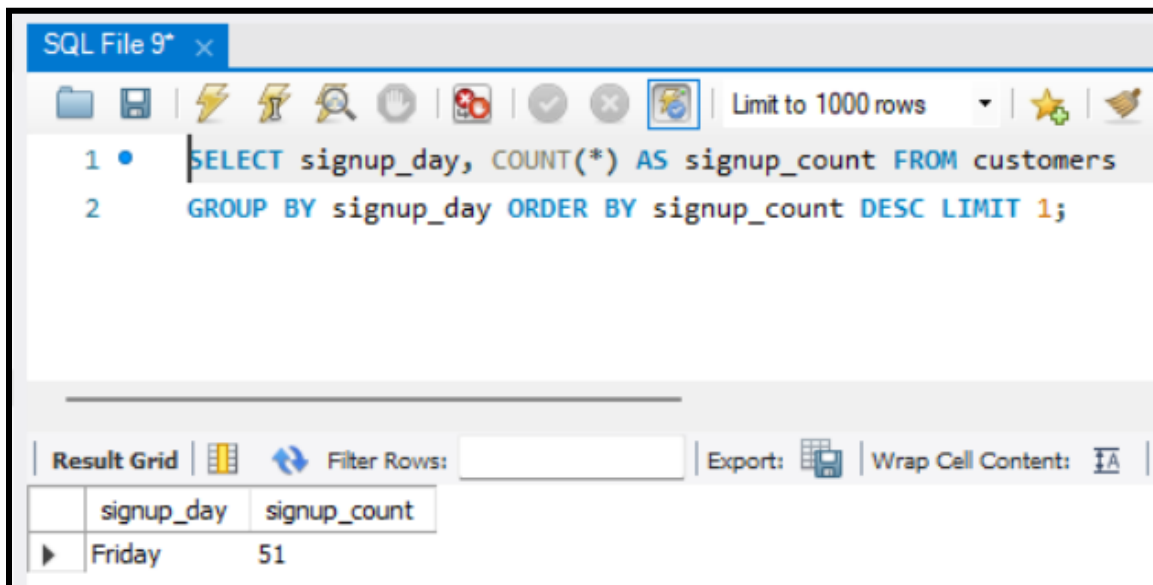


The screenshot shows a SQL editor window titled "SQL File 9". The SQL commands are as follows:

```
1 • ALTER TABLE customers ADD COLUMN signup_day VARCHAR(20);
2 • UPDATE customers SET signup_day = DAYNAME(signup_date_new);
3 • SELECT signup_date_new, signup_day FROM customers LIMIT 10;
```

The result grid below shows the output of the third command, displaying the first 10 rows of the 'customers' table with columns 'signup_date_new' and 'signup_day'.

signup_date_new	signup_day
2023-04-21	Friday
2024-06-11	Tuesday
2024-06-02	Sunday
2025-03-24	Monday
2024-07-07	Sunday
2024-11-16	Saturday
2023-10-31	Tuesday
2024-02-08	Thursday
2025-04-05	Saturday
2023-11-24	Friday



The screenshot shows a SQL editor window titled "SQL File 9". The SQL commands are as follows:

```
1 • SELECT signup_day, COUNT(*) AS signup_count FROM customers
2   GROUP BY signup_day ORDER BY signup_count DESC LIMIT 1;
```

The result grid below shows the output of the second command, displaying the day with the highest number of signups.

signup_day	signup_count
Friday	51